

บทที่ 7

การออกแบบ

โครงการนี้ เป็นการสร้างฮาร์ดแวร์ (Hardware) ต้นแบบที่มีการนำเอาชิป (Chip) มาใช้ในการเข้าและถอดรหัส

7.1 แนวคิดในการออกแบบ

การออกแบบชุดต้นแบบการเข้ารหัสด้วยฮาร์ดแวร์ จะเริ่มด้วยการมองภาพรวมของการทำงานทั้งหมดของระบบ หลังจากนั้นจะทำการศึกษาทฤษฎีต่าง ๆ ที่เกี่ยวข้องจนเข้าใจความต้องการทั้งหมดของระบบแล้ว เราจึงเริ่มทำการออกแบบระบบรวมทั้งหมด โดยในการออกแบบนั้น จะทำการแบ่งระบบออกเป็นส่วนย่อย ๆ โดยในแต่ละส่วนจะถูกออกแบบให้มีส่วนติดต่อที่เหมือนกับส่วนติดต่อมาตรฐานให้มากที่สุด เพื่อความสามารถในการใช้งานร่วมกันได้กับระบบที่มีอยู่แล้วในปัจจุบัน และความสามารถในการนำเอาส่วนต่าง ๆ กลับมาใช้ได้อีก รวมถึงเพื่อลดภาระของงานที่จะใช้ในการแก้ปัญหาคล่องตัว

7.2 ภาพรวมในการออกแบบ

ในการออกแบบ เราจะยึดเอาการติดต่อระหว่างส่วนต่าง ๆ เป็นหลัก ซึ่งทำให้เราแบ่งส่วนต่าง ๆ ออกมาเป็น 3 ส่วนใหญ่ ๆ คือ ส่วนของอุปกรณ์ฝังฮาร์ดแวร์ ซอฟต์แวร์ดีไวซ์ไดรเวอร์ (Software Device Driver) และซอฟต์แวร์วีพีเอ็น (Software VPN) โดยฮาร์ดแวร์และซอฟต์แวร์ทั้ง 2 ส่วน จะทำงานร่วมกันโดยผ่านบริการของระบบปฏิบัติการ (OS Services)

ส่วนของฮาร์ดแวร์จะทำการติดต่อกับไอเอสเอบัส (ISA Bus) โดยมีการ์ดไอเอสเอ (ISA Card) เป็นส่วนหน้าในการเชื่อมต่อของฝังฮาร์ดแวร์ โดยการทำงานส่วนใหญ่ในฝั่งนี้จะขึ้นอยู่กับบอร์ดเอฟพีจีเอ (FPGA Board) เป็นหลัก ซึ่งการเข้ารหัสของข้อมูลจะถูกกระทำในส่วนของฝังฮาร์ดแวร์นี้ รวมถึงการควบคุมการติดต่อกับไอเอสเอบัสด้วย ในรุ่นแรกนี้ เราจะยังไม่ได้ใช้ความสามารถทั้งหมดของบัส เนื่องจากข้อจำกัดของเวลา เนื่องจากการพัฒนาในส่วนของฮาร์ดแวร์จะต้องใช้เวลานานกว่าการพัฒนาในส่วนซอฟต์แวร์

สำหรับส่วนของการเข้ารหัสที่อยู่ในฝั่งฮาร์ดแวร์นั้น จะใช้การเข้ารหัสแบบ XOR เนื่องจากข้อจำกัดของบอร์ดเอฟพีจีเอที่มีขนาดเล็ก แต่ผู้พัฒนาเองก็ได้ทำในส่วนของการเข้ารหัสที่ใช้วิธีการเข้ารหัสแบบ DES เอาไว้แล้วเช่นกัน

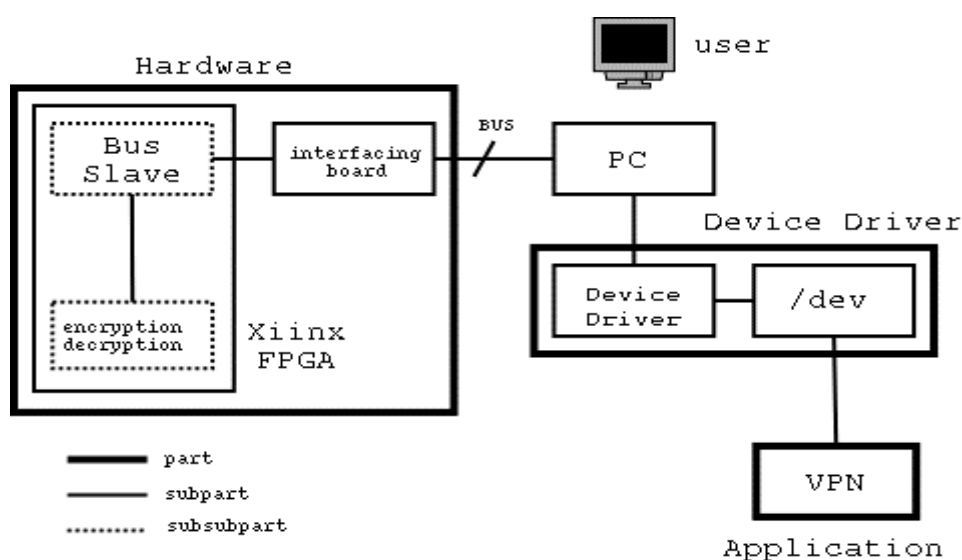
ส่วนของซอฟต์แวร์ดีไวซ์ไดรเวอร์ จะทำงานโดยฝังตัวเข้ากับบริการของระบบปฏิบัติการ โดยจะอาศัยบริการหลัก (กลาง) ของระบบปฏิบัติการทำงาน และสร้างบริการของระบบปฏิบัติการขึ้นมาใหม่ชุดหนึ่ง เพื่อใช้ในการเข้าถึงอุปกรณ์ฝังฮาร์ดแวร์ รวมถึงเพื่อให้บริการแก่โปรแกรมใช้งานทั่วไปให้สามารถเรียกใช้งานได้ด้วย ซึ่งในชุดทดลองนี้ก็คือโปรแกรมวีพีเอ็นนั่นเอง สำหรับระบบปฏิบัติการที่เลือกมาใช้

จะเป็นระบบปฏิบัติการ Linux ที่ Kernel รุ่น 2.4.0 มาตรฐาน สาเหตุที่เลือกนำระบบปฏิบัติการตัวนี้มาใช้ เพราะเห็นถึงความง่ายในการพัฒนาและรวมถึงชุดพัฒนาที่สามารถนำมาใช้ได้ง่าย

อีกส่วนหนึ่งของซอฟต์แวร์ คือ โปรแกรมวิพีเอ็น ที่เรานำมาดัดแปลงเพื่อให้มาเรียกใช้งานตัวดีไวส์ไดรเวอร์อีกที ในส่วนของโปรแกรมวิพีเอ็นนี้ เราได้เลือกใช้โปรแกรมวิพีเอ็นที่มีอยู่แล้ว ซึ่งเป็นซอฟต์แวร์เสรี เพื่อความง่ายและรวดเร็วในการพัฒนา อันเกิดจากความไม่ชำนาญในการทำงาน สำหรับสาเหตุที่เราทำการแบ่งส่วนของโปรแกรมวิพีเอ็นออกมาจากตัวซอฟต์แวร์วิพีเอ็นอีกทีหนึ่ง เนื่องจากเวลาทำการเรียกใช้งานส่วนของโปรแกรมวิพีเอ็นกับส่วนของซอฟต์แวร์ดีไวส์ไดรเวอร์นั้น ยังต้องติดต่อกันและกันโดยผ่านส่วนติดต่อมาตรฐานของระบบปฏิบัติการอีกที รวมถึงในด้านสถานะแวดล้อมของการใช้งาน โปรแกรมดีไวส์ไดรเวอร์จะรันในเคอร์เนลโหมด ส่วนโปรแกรมวิพีเอ็นจะรันในยูสเซอร์โหมด

7.3 การทำงานร่วมกันในแต่ละส่วน

ส่วนต่างๆ ของชุดต้นแบบนี้จะติดต่อซึ่งกันและกันตามรูปด้านล่าง ในระบบของชุดต้นแบบจะไม่มีการติดต่อถึงกันโดยตรงระหว่างส่วนต่าง ๆ นอกเหนือจากที่แสดงไว้ในรูปนี้



รูปที่ 7-1: การทำงานร่วมกันระหว่างแต่ละส่วนของระบบ

จากรูป จะเห็นส่วนของระบบถูกแบ่งออกเป็น 3 ส่วน ตามที่ได้กล่าวเอาไว้ก่อนหน้านี้แล้ว

7.4 การออกแบบในแต่ละส่วนย่อย

7.4.1 การออกแบบส่วนของอุปกรณ์ฮาร์ดแวร์

เนื่องจากการทำงานโดยใช้ซอฟต์แวร์นั้น ยังมีขีดจำกัดในเรื่องความเร็วในการส่งข้อมูล และความยืดหยุ่นในการนำไปประยุกต์ใช้งานในสภาพการทำงานปกติ ดังนั้นโครงการนี้จึงได้นำเอาการเข้ารหัสด้วยฮาร์ดแวร์เข้ามาช่วยแก้ปัญหาในส่วนนี้ และต้นแบบในโครงการนี้ยัง

สามารถนำไปพัฒนาต่อให้ใช้งานได้เป็นอิสระโดยไม่ต้องพึ่งพาเครื่องคอมพิวเตอร์ส่วนบุคคลในการทำงานได้ด้วย

ในส่วนของอุปกรณ์ฮาร์ดแวร์ จะประกอบไปด้วยอุปกรณ์ 2 ชิ้น คือ ส่วนของการ์ดไอเอสเอ และส่วนของบอร์ดเอฟพีจีเอ

7.4.1.1 ส่วนของการ์ดไอเอสเอ

ส่วนของการ์ดไอเอสเอนี้ จะทำหน้าที่ป้องกันการชนกันของสัญญาณไฟฟ้าจากไอเอสเอบัสและบอร์ดเอฟพีจีเอ ซึ่งในการทำงานของส่วนนี้จะมีอะไรมา โดยจะทำหน้าที่รับสัญญาณจากส่วนควบคุมการติดต่อกับไอเอสเอบัสในบอร์ดเอฟพีจีเอ ซึ่งเป็นสัญญาณการสั่งให้บอร์ดทำงาน โดยทำการส่งสัญญาณออกไปยังไอเอสเอบัส ในกรณีที่เป็นสัญญาณที่ต้องใช้ช่องทางร่วมกับอุปกรณ์ตัวอื่นในไอเอสเอบัส และการกำหนดทิศทางการรับส่งข้อมูล ในกรณีที่ข้อมูลเป็นแบบ 2 ทิศทาง เช่น ข้อมูลในคาต้าบัส (Data Bus)

7.4.1.2 ส่วนของบอร์ดเอฟพีจีเอ

ส่วนของบอร์ดเอฟพีจีเอนี้ จะติดต่อโดยตรงกับการ์ดไอเอสเอ และทำการรับส่งข้อมูลทั้งหมดผ่านทางการ์ดไอเอสเอ การติดต่อจะใช้การเดินสายลอยธรรมดา โดยมีที่เชื่อมจุดศักย์ไฟฟ้าศูนย์ของบอร์ดเอฟพีจีเอเข้ากับจุดศักย์ไฟฟ้าศูนย์ของคอมพิวเตอร์ โดยผ่านทางเครื่องพีซี โดยบอร์ดเอฟพีจีเอที่เลือกนำมาใช้จะเป็นรุ่น XC4010E ซึ่งเป็นรุ่นที่ใช้กันในการเรียนของภาควิชา

ในบอร์ดเอฟพีจีเอนี้ จะถูกโปรแกรมให้ทำหน้าที่เป็นตัวควบคุมการติดต่อกับไอเอสเอบัส และทำหน้าที่ในส่วนของการเข้ารหัส ซึ่งในโปรแกรมที่เราเขียนมานั้น ได้แบ่งออกเป็น 2 ส่วน คือ ส่วนของชุดโปรแกรมการเข้ารหัสและโปรแกรมที่เชื่อมต่อระหว่างการ์ดไอเอสเอและชุดโปรแกรมการเข้ารหัส

ซึ่งในโอกาสหน้า ถ้าเรามีบอร์ดเอฟพีจีเอที่มีขนาดใหญ่กว่านี้ เราอาจจะถอดเอาชุดโปรแกรมการเข้ารหัสตัวปัจจุบันออกและแทนด้วยเข้ารหัสชุดใหม่ โดยไม่จำเป็นต้องแก้ส่วนที่ติดต่อกับไอเอสเอบัส รวมถึงยังสามารถแก้ส่วนที่ติดต่อกับไอเอสเอบัสเพื่อเปลี่ยนเป็นบัสนชนิดอื่นได้โดยไม่กระทบกับส่วนของการเข้ารหัส

7.4.1.3 ส่วนของโปรแกรมที่โหลดลงบอร์ดเอฟพีจีเอ

ในส่วนของโปรแกรมที่ทำการโหลดลงบนบอร์ดเอฟพีจีเอนี้ ถูกเขียนด้วยภาษาวีเอสดีแอล ซึ่งจากโค้ดของโปรแกรมที่เขียนขึ้นนั้น จะมีส่วนที่ติดต่อกับด้านการ์ดไอเอสเอ ดังแสดงในโค้ดที่ตัดมาด้านล่างนี้

```
port( sys_clk      :in std_logic;
      sys_reset    :in std_logic;
```

```

isa_aen      :in std_logic;

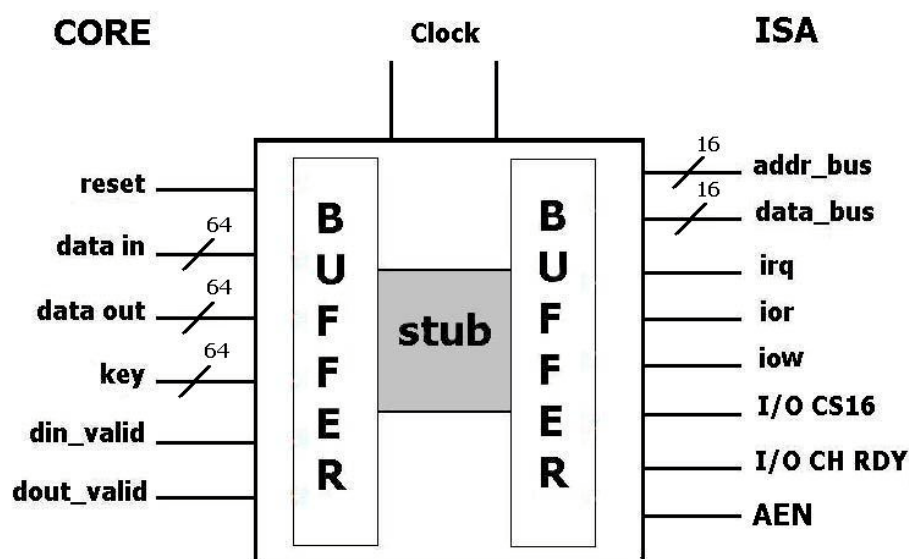
cs_out       :out std_logic;
dir_out      :out std_logic;

addr_bus     :in std_logic_vector(11 downto 0);
data_bus     :inout std_logic_vector(15 downto 0);
irq          :out std_logic := '1';
ior          :in std_logic;
iow          :in std_logic;
iocs16       :out std_logic;
iochrdy      :out std_logic
);

```

พอร์ต cs_out และ dir_out ใช้เพื่อควบคุมการทำงาน และควบคุมทิศทางของสัญญาณภายในการ์ดไอเอสเอ ส่วนสัญญาณที่เหลือจะเป็นสัญญาณของไอเอสเอบัสที่ถูกควบคุมด้วยการ์ดไอเอสเอ

ส่วนโปรแกรมที่ใช้ในการเชื่อมต่อระหว่างการ์ดไอเอสเอและชุดโปรแกรมเข้ารหัส คือ โปรแกรมสตัป โดยโปรแกรมสตัปจะมีส่วนเชื่อมต่อและโครงสร้าง ตามที่แสดงให้เห็นในรูปด้านล่างนี้



รูปที่ 7-2: ส่วนเชื่อมต่อและโครงสร้างของสตัป

โดยด้านซ้ายมือเป็นส่วนของชุดโปรแกรมเข้ารหัส และส่วนด้านขวามือคือ ไอเอสเอบัส ซึ่งเราสามารถปรับเปลี่ยนชุดโปรแกรมการเข้ารหัสได้โดยที่ไม่จำเป็นต้องแก้ไขตัวโปรแกรมสตัป ถ้าหากโปรแกรมการเข้ารหัสนั้นมีความกว้างของข้อมูลและกฎเกณฑ์เท่ากับชุดโปรแกรมเข้ารหัสของเรา

การทำงานของอุปกรณ์ฮาร์ดแวร์ของเรา จะเป็นลักษณะพอร์ตเบส (port-based) ดีไวซ์ โดยแอดเดรส (Address) ที่เราใช้จะอยู่ในช่องของ 0x0110 ถึง 0x012F โดยแบ่งออกเป็น 3 ช่วง คือ

- แอดเดรสช่วงแรก (0x0110 ถึง 0x0117) ใช้สำหรับรับข้อมูลที่จะทำการเข้ารหัสจากไอเอสเอบีเอส
- แอดเดรสช่วงที่ 2 (0x0118 ถึง 0x011F) สำหรับใช้รับข้อมูลที่เป็นคีย์ในการเข้ารหัส
- และช่วงสุดท้าย (0x0120 ถึง 0x0130) จะเป็นข้อมูลที่ผ่านการเข้ารหัสแล้ว โดยการรับและส่งข้อมูล จะใช้การรับส่งข้อมูลทีละ 2 ไบต์ (ไบต์ละ 8 บิต) และการรับ-ส่งจะใช้แอดเดรสทีละ 2 ช่วง เพื่อให้สามารถนำไปดัดแปลงจากการทำงานด้วยพอร์ตเป็นเมโมรีแมป (Memory -Mapped) แทนได้ง่ายยิ่งขึ้น

7.4.2 ส่วนของซอฟต์แวร์ดีไวซ์ไครเวอร์

ส่วนของซอฟต์แวร์ดีไวซ์ไครเวอร์นั้นถูกโปรแกรมออกมาเพื่อทำหน้าที่ 2 อย่าง คือ การควบคุมข้อกำหนดในการใช้งานอุปกรณ์ และสร้างบริการสำหรับการใช้งานอุปกรณ์จากโปรแกรมผู้ใช้งานทั่วไป

แต่ซอฟต์แวร์ดีไวซ์ไครเวอร์ยังแบ่งส่วนติดต่อได้เป็น 2 ฟังก์ชัน คือ ส่วนติดต่อกับอุปกรณ์ และส่วนติดต่อผู้ใช้งาน ในส่วนติดต่อกับอุปกรณ์นั้น เราจะให้บริการในระดับล่างของระบบปฏิบัติการเพื่อจองทรัพยากร และเพื่อรับส่งข้อมูลกับอุปกรณ์ฮาร์ดแวร์ เช่น การจองพอร์ตแอดเดรสของอุปกรณ์ และการส่งข้อมูลออกทางพอร์ต เป็นต้น สำหรับส่วนติดต่อผู้ใช้งานนั้น จะให้บริการในระดับบนเพื่อเพิ่มบริการของตัวเองเข้ากับบริการที่มีอยู่ของระบบปฏิบัติการ การทำงานหลัก ๆ ของไครเวอร์ คือ การควบคุมการรับส่งข้อมูลระหว่างส่วนติดต่อทั้ง 2 ฟังก์ชัน

สำหรับส่วนติดต่อกับผู้ใช้งานนั้น เราจะเลือกใช้การให้บริการโดยมองอุปกรณ์เสมือนเป็นแฟ้มข้อมูลธรรมดา (Device File) ซึ่งในการใช้งานนั้น โปรแกรมฝั่งผู้ใช้งานสามารถเปิดดีไวซ์ไฟล์ได้โดยผ่านทางฟังก์ชัน (Function) `open()` และสามารถรับส่งข้อมูลเพื่อทำการเข้ารหัสได้โดยผ่านทางฟังก์ชัน `read()` และ `write()` รวมถึงสามารถควบคุมการใช้งานอื่น ๆ ได้โดยผ่านทางฟังก์ชัน `ioctl()` ซึ่งในตัวติดต่อกับผู้ใช้งานนั้น จะให้บริการเฉพาะฟังก์ชันที่จำเป็นสำหรับการใช้งานในเบื้องต้นนี้เท่านั้น โดยในซอฟต์แวร์ดีไวซ์ไครเวอร์ของเรานั้นจะรองรับฟังก์ชันตามการประกาศโครงสร้างต่อไปนี้

```
static struct file_operations mind_fops = {
    read:    mind_read,
    write:   mind_write,
    ioctl:   mind_ioctl,
    open:    mind_open,
    release: mind_release,
};
```

การควบคุมการติดต่อสื่อสารนั้น เราจะต้องสามารถรับประกันความถูกต้องของข้อมูลได้ด้วย ซึ่งจะต้องรับประกันได้ว่า ถ้าโปรแกรมสามารถ write() หรือ read() ได้ โดยไม่ได้รับการแจ้งเตือน (return) ความผิดพลาด ดังนั้นข้อมูลที่ได้ทำการรับส่งหรือทำการประมวลผลกับดีไวซ์ไครเวอร์นั้น ก็จะต้องเป็นข้อมูลที่มีความถูกต้องด้วย ซึ่งสิ่งสำคัญสำหรับการนี้คือ การป้องกันการเรียกโปรแกรมซ้อนทับ โดยเฉพาะอย่างยิ่ง การเขียนทับที่ตำแหน่งเดียวกัน ซึ่งในประเด็นนี้เราจึงแก้ปัญหาโดยใช้ซีมาฟอว์ (Semaphore) มาช่วยในการป้องกันการรันฟังก์ชันซ้อนทับกัน โดยเราได้ใช้ซีมาฟอว์ทั้งหมด 3 ตัว ดังตัวอย่างโค้ด (Code) ด้านล่าง

```
static struct semaphore mind_read_sem;    // mutex read()
static struct semaphore mind_write_sem;   // mutex write()
static struct semaphore mind_ioctl_sem;   // mutex key and port
```

โดยเราได้ทำการเริ่มต้นเซต (Set) ค่าทั้งหมดของซีมาฟอว์ในตอนเริ่มต้นของการทำงานของดีไวซ์ไครเวอร์ นอกจากนั้น ยังมีการรับประกันในขณะที่มีการเปลี่ยนแปลงข้อมูลที่มีความสำคัญเป็นพิเศษ เช่น ข้อมูลที่ใช้ในการควบคุมและการนับของโปรแกรมดีไวซ์ไครเวอร์โดยใช้สปินล็อก (spinlock)

ดีไวซ์ไครเวอร์จะถูกออกแบบมาโดยเน้นที่ความสามารถในการปรับแต่งได้ โดยซอสโค้ด (Source Code) ของโปรแกรมดีไวซ์ไครเวอร์จะประกอบขึ้นมาจาก 2 ไฟล์ คือ ไฟล์ชื่อ mind.c และ mind.h โดยทั้ง 2 ไฟล์นี้ สามารถสร้างดีไวซ์ไครเวอร์ได้ 2 รูปแบบการทำงาน คือ แบบแรกจะเป็นดีไวซ์ไครเวอร์ที่จะทำการเรียกใช้อุปกรณ์เข้ารหัส และแบบที่สองจะเป็นดีไวซ์ไครเวอร์ที่จะทำการเรียกใช้อีมูเลเตอร์ (Emulator) ที่รวมอยู่ในดีไวซ์ไครเวอร์เอง

สาเหตุที่ทำให้ดีไวซ์ไครเวอร์ออกมา 2 ชุด เนื่องจากว่าในการทำงานของโปรแกรมวิทีเอ็นเราต้องมีเกตเวย์ (Gateway) 2 ฟังก์ชัน แต่เนื่องจากเรามีอุปกรณ์ฮาร์ดแวร์ชุดเดียว เราจึงได้ทำดีไวซ์ไครเวอร์ออกมา 2 ชุด อีกทั้งการที่ฟังก์ชันหนึ่งเป็นฮาร์ดแวร์จึงสามารถทำงานร่วมกับโปรแกรมที่เป็นซอฟต์แวร์อีมูเลเตอร์ได้ ซึ่งสามารถแสดงให้เห็นถึงความเป็นมอดูลาร์ (Modular) ของระบบของเรา

นอกจากความสามารถในการปรับแต่งการทำงานแล้ว เรายังสามารถกำหนดระดับของการแสดงข้อความดีบั๊ก (Debug) ได้ โดยการส่งผ่านระดับของการดีบั๊กให้กับตัวคอมไพเลอร์ (Compiler) หรือสามารถกำหนดระดับของการดีบั๊กในซอสโค้ดเองได้ ความสามารถในการปรับแต่งทั้งหมดนี้จะอาศัยคอมไพเลอร์ เพื่อทำ preprocessing คือ เพื่อเลือกเอาชุดของคำสั่งที่ถูกต้องมาคอมไพล์ (Compile) เป็นดีไวซ์ไครเวอร์ ซึ่งการคอมไพล์ดีไวซ์ไครเวอร์จะสามารถทำได้โดยใช้โปรแกรม make ซึ่งในตำแหน่งที่เก็บซอสโค้ดของดีไวซ์ไครเวอร์จะมีไฟล์สคริปต์ (Script

File) ชื่อ “Makefile” มาให้ด้วย โดยกฎที่สำคัญที่สุดของการเมกไฟล์ (Makefile) คือ เป็นตัวกำหนดชนิดของดีไวซ์ไครเวอร์ ดังนี้

```
mind.o: mind.c mind.h ldd.h
    gcc -Wall -c -D__KERNEL__ -DMODULE -D__NO_VERSION__ -O2 mind.c \
        -o mind.o
no_mind.o: mind.c mind.h ldd.h
    gcc -Wall -c -DNO_MIND -D__KERNEL__ -DMODULE -D__NO_VERSION__ -O2 \
        mind.c -o no_mind.o
```

จะเห็นได้ว่าสิ่งที่ต่างกันคือ ในกระบวนการสร้าง mind.o จะมีการประกาศมาโคร (Macro) ชื่อ NO_MIND ไว้ด้วย

7.4.3 ส่วนของโปรแกรมวิพีเอ็น

ส่วนของโปรแกรมวิพีเอ็นนี้ เราได้ทำการดัดแปลงส่วนของการเข้ารหัสของโปรแกรม รวมถึงการเพิ่มส่วนของการควบคุมข้อกำหนดในการใช้งานอุปกรณ์เข้าไปในตัวโปรแกรมด้วย ในการดัดแปลงนั้น เราได้เปลี่ยนส่วนเข้ารหัสของตัวเดิม โดยเราได้ลบตัวเข้ารหัสเดิมออก แล้วให้มาเรียกการเข้ารหัสโดยใช้ฮาร์ดแวร์เข้ารหัสผ่านทางซอฟต์แวร์ดีไวซ์ไครเวอร์แทน โดยเราจะทำการเปิดดีไวซ์ไฟล์ในตอนต้นโปรแกรมเพื่อว่า เมื่อเราทำการแตกโพรเซส (fork) เพื่อรองรับการทำงานซ้อนกันหลาย ๆ เซสชัน (Session) แล้ว เราจะได้ไม่จำเป็นต้องเปิดไฟล์อีก

เราเริ่มต้นเพื่อเตรียมการเข้ารหัสได้ง่าย ๆ ด้วยโค้ดที่นำมาแสดงนี้

```
if ((fdesc = open("/dev/misc/isagcrypt/mind", O_RDWR)) < 0) {
    syslog(LOG_ERR, "Can't open /dev/misc/isagcrypt/mind");
    return fdesc; }
else {
    syslog(LOG_INFO, "Opened /dev/misc/isagcrypt/mind");
    close(fdesc_r);
}
```

ในการแก้ไขโปรแกรมของเราในครั้งนี้ จะไม่กระทบกับการทำงานส่วนเดิมอื่น ๆ ของตัวโปรแกรมวิพีเอ็น ซึ่งทำให้เราสามารถทำการถอดฟังก์ชันการเข้ารหัสเข้าออกได้โดยไม่ต้องทำการคอมไพล์ตัวโปรแกรมวิพีเอ็นใหม่ และในการเปลี่ยนมาใช้อุปกรณ์เข้ารหัสของเรานั้น เราจะต้องป้องกันความถูกต้องของข้อมูลด้วย เพราะระบบการเข้ารหัสของเรายังเป็นระบบเดี่ยวแบบวงทางเดียว ซึ่งทำให้ไม่สามารถทำการเข้ารหัสแบบขนานพร้อม ๆ กันทีละหลายชุดได้ แต่ในการทำงานจริงแล้ว เราจะต้องทำการประมวลผลให้แต่ละเซสชันทำงานไปพร้อม ๆ กัน ในกรณีนี้ เราได้นำเอาการใช้ SysV Semaphore มาช่วย โดยทำการร้องขอซีมาฟอร์จากระบบปฏิบัติการเพื่อใช้ร่วมกันระหว่างโพรเซส (Process)

โปรแกรมวิพีเอ็นที่เราใช้นี้ ยังมีข้อจำกัดในการทำงานหลายอย่าง เช่น ยังไม่มีระบบการแลกเปลี่ยนกุญแจในการเข้ารหัส และการเชื่อมต่อยังเป็นแบบสแตติกไคลเอนต์เซิร์ฟเวอร์ (Static

Client Server) คือ เกตเวย์ฝั่งหนึ่งจะต้องเปิดรอการเชื่อมต่อ และอีกฝั่งหนึ่งจะต้องร้องขอการเชื่อมต่อเข้าไป ซึ่งถือว่าเป็นความไม่ยืดหยุ่นในการนำไปใช้งาน ตัวอย่างการคอนฟิกกูเรชัน (Configuration) ไฟล์ของโปรแกรมวีพีเอ็น

```
# TUN example. Session 'cobra'.
cobra {
    pass Ma&^TU;          # Password
    type tun;              # IP tunnel
    proto udp;             # UDP protocol
    comp lzo:9;            # LZO compression level 9
    encr yes;              # Encryption
    keepalive yes;         # Keep connection alive

    up {
        # Connection is Up
        # 10.3.0.1 - local, 10.3.0.2 - remote
        ifconfig "% 10.3.0.1 pointopoint 10.3.0.2 mtu 1450";
    };
}
```

7.5 แนวคิดในการออกแบบชุดทดสอบ

ในการทำงานครั้งนี้ เราได้ออกแบบให้มีลักษณะเป็นมอดูลาร์ให้มากที่สุด ดังนั้นแต่ละส่วนจะมีความทำงานเป็นอิสระ ซึ่งจะช่วยให้เป็นอย่างมากในเรื่องของการนำเอาส่วนต่าง ๆ กลับมาใช้ได้อีกในงานต่อไป และในการทำงานครั้งนี้ เราได้ออกแบบชุดทดลองออกมาเป็น 2 ชุดเพื่อทำการทดลองส่วนของอุปกรณ์ฮาร์ดแวร์ และส่วนของโปรแกรมดีไวซ์ไครเวอร์ ในชุดทดลองสำหรับแต่ละส่วนนี้จะทดลองการทำงานของส่วนนั้น ๆ ได้อย่างสมบูรณ์โดยไม่ต้องอาศัยส่วนอื่น ๆ ของระบบช่วยในการทดลอง

7.6 ชุดทดสอบส่วนอุปกรณ์ฮาร์ดแวร์

ในชุดทดลองฮาร์ดแวร์นั้น เราจะใช้การทดลองด้วยอินพุตเวกเตอร์ (Input Vector) โดยแบ่งออกเป็น 2 ชุดย่อย ชุดแรกเป็นการทดลองโดยใช้ค่าตายตัว โดยอิงเลือกค่าต่อไปนี้เป็นค่าหลัก 0x0000000000000000, 0x0123456789ABCDEF, 0xFEDCBA9876543210 และ 0xFFFFFFFFFFFFFFFF และทำการจับคู่กันทั้งหมดในแซมเปิลสเปซ (Sample Space) แล้วนำมาเทียบกับค่าที่คาดหมายว่าจะได้รับ ส่วนชุดทดลองหลังนั้น เราจะทำการสุ่มเลือกค่าระหว่าง 0x0000000000000000 ถึง 0xFFFFFFFFFFFFFFFF มาจำนวนหนึ่งและทำการทดลองการเข้ารหัสและเทียบกับโปรแกรมเข้ารหัสที่เขียนไว้ในชุดทดลอง โดยโปรแกรมทดสอบทั้ง 2 ชุดนั้น จะเขียนด้วยภาษาแอสเซมบลี และทำการทดสอบบนระบบปฏิบัติการไมโครซอฟท์ดอส (Microsoft DOS)

ในการทำงานในส่วน of อุปกรณ์ฮาร์ดแวร์นี้ยังมีการทดสอบเฉพาะกิจด้วยอุปกรณ์วัดค่าลอจิก (Logic) บางประเภทเช่น ลอจิกโพรบ (Logic Probe) และ ดิจิตอลออสซิลโลสโคป (Digital Oscilloscope) เพื่อทำการตรวจสอบการทำงานจริงในส่วน of ตัวฮาร์ดแวร์ และ รวมถึงการป้อนค่าไปที่บอร์ดเอฟพีจีเอ

เพื่อทดสอบการทำงานเฉพาะบางอย่าง แต่ในการทดลองส่วนนี้เป็นการทดลองเฉพาะกิจ ซึ่งจะไม่ขอกล่าวรายละเอียดในที่นี้

7.7 ชุดทดสอบส่วนของระบบดีไวซ์ไครเวอร์

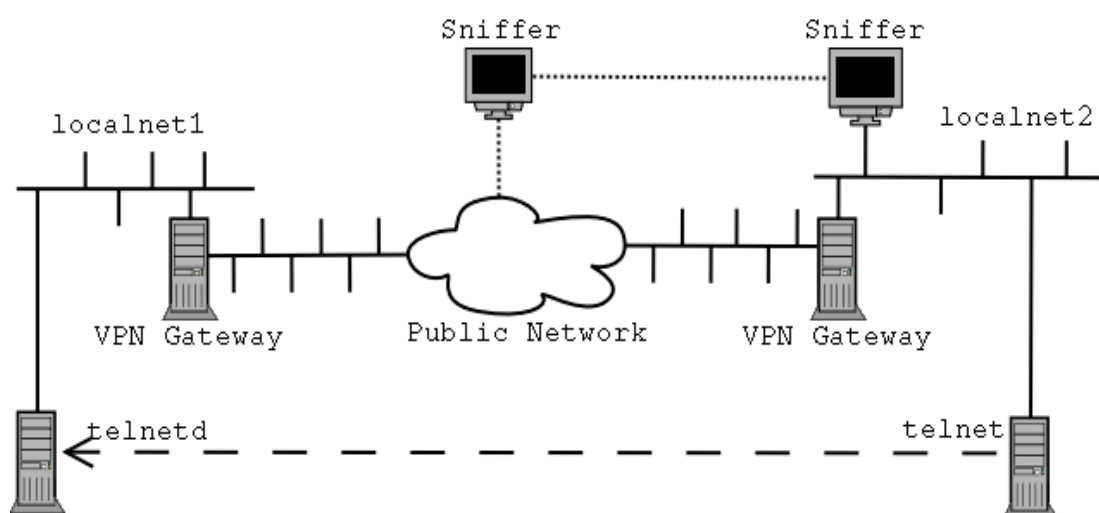
จากการออกแบบ ตัวดีไวซ์ไครเวอร์จะเน้นที่ความยืดหยุ่นในการนำไปทำงาน โดยโปรแกรมดีไวซ์ไครเวอร์ถูกออกแบบให้สามารถทำงานได้โดยไม่ต้องใช้อุปกรณ์ฮาร์ดแวร์ด้วย เป็นผลให้ ถ้าเราสามารถมั่นใจได้ว่า ดีไวซ์ไครเวอร์แบบที่ไม่อาศัยฮาร์ดแวร์สามารถทำงานได้ถูกต้องแล้ว เราก็สามารถนำเอาดีไวซ์ไครเวอร์ชุดนี้ไปทดลองกับดีไวซ์ไครเวอร์ในเวอร์ชันที่ทำงานกับอุปกรณ์ฮาร์ดแวร์ได้

ในการพัฒนา เราได้ทำดีไวซ์ไครเวอร์แบบที่ไม่ต้องใช้ฮาร์ดแวร์ขึ้นมาก่อน และทำการทดสอบดีไวซ์ไครเวอร์ชุดนี้ด้วยอินพุตเวกเตอร์เดียวกับที่ใช้ในการทดสอบส่วนอุปกรณ์ฮาร์ดแวร์ แต่เขียนขึ้นมาใหม่ด้วยภาษาซีเพื่อความสะดวกในการใช้งานและทำความเข้าใจ จนเมื่อเรามั่นใจดีไวซ์ไครเวอร์ชุดนี้แล้ว เราจะสามารถนำดีไวซ์ไครเวอร์ชุดนี้เป็นฐานในการทดสอบดีไวซ์ไครเวอร์ในรูปแบบที่ใช้อุปกรณ์ฮาร์ดแวร์ได้ต่อไป

หลังจากที่เราทำดีไวซ์ไครเวอร์เวอร์ชัน (Version) แรกเสร็จเรียบร้อยแล้ว ก็จะเริ่มทำงานปรับแก้ดีไวซ์ไครเวอร์ตัวเดิมให้มาติดต่อกับอุปกรณ์ฮาร์ดแวร์เพื่อทำการเข้ารหัส แทนการใช้การเข้ารหัสภายในตัวดีไวซ์ไครเวอร์ในแบบเดิม และทำการทดสอบดีไวซ์ไครเวอร์ชุดนี้ โดยในการทดสอบดีไวซ์ไครเวอร์ชุดนี้ จะเท่ากับการทดสอบการทำงานร่วมกันระหว่างอุปกรณ์ฮาร์ดแวร์เข้ารหัสร่วมกับดีไวซ์ไครเวอร์ โดยใช้ดีไวซ์ไครเวอร์ในเวอร์ชันแรกที่ไม่ต้องการฮาร์ดแวร์มาเป็นตัวเปรียบเทียบ

7.8 ชุดทดสอบโปรแกรมวิพีเอ็น

ในการทดสอบโปรแกรมนี้ จะใช้การทดสอบจากการใช้งานจริง โดยออกแบบสภาพแวดล้อมให้เหมือนในการใช้งานจริง ดังแสดงในรูปด้านล่าง



รูปที่ 7-3: รูปแบบการเชื่อมต่อเพื่อทดสอบโปรแกรมวิพีเอ็น

โดยวีพีเอ็นเกตเวย์ฝั่งหนึ่งจะใช้โปรแกรมวีพีเอ็นร่วมกับดีไวซ์ไครเวอร์ที่ไม่จำเป็นต้องใช้ฮาร์ดแวร์ ส่วนวีพีเอ็นเกตเวย์อีกฝั่งหนึ่งจะเป็นการทำงานร่วมกันระหว่างโปรแกรมวีพีเอ็นร่วมกับดีไวซ์ไครเวอร์และอุปกรณ์ฮาร์ดแวร์เข้ารหัส

ในช่วงของการทดสอบนั้น อาจจะต้องมีการทดสอบผลกระทบของการปรับแก้โปรแกรมวีพีเอ็นด้วยการปรับแต่งการตั้งค่าในไฟล์คอนฟิกูเรชันของโปรแกรมวีพีเอ็น ในการทดสอบวีพีเอ็นนี้ ถือเป็นการทดสอบใหญ่ครั้งสุดท้าย โดยเป็นการทดสอบการรวมส่วนต่าง ๆ ของระบบ และเป็นการทดสอบการทำงานร่วมกัน รวมถึงเป็นการทดสอบการใช้งานจริง โดยหลังจากที่ระบบสามารถรันได้สมบูรณ์โดยไม่มี ความผิดพลาดภายในตัวระบบเองแล้ว เราก็จะทดสอบครั้งสุดท้าย โดยใช้การ sniff (Sniff) ข้อมูลที่ติดต่อ ผ่านการเทลเน็ต (Telnet) ซึ่งเป็นข้อมูลแบบข้อความล้วน ๆ ที่สามารถอ่านเข้าใจได้บางส่วน ถ้าเราเปิดการใช้งานดีไวซ์ไครเวอร์ทั้ง 2 ฝั่งและ 2 เวอร์ชัน จากทั้ง 2 โปรแกรมวีพีเอ็นในเกตเวย์ทั้ง 2 ตัวแล้ว เราจะสามารถทำการปกปิดข้อมูลได้ ซึ่งถือว่า ระบบสามารถทำงานได้สมบูรณ์ในระดับของชุดต้นแบบ